

VIKUNA TECHNOLOGIES

VaNi Agent Topology & Memory Budget

Architecture Note v1.1

How agents are split · How memory is tiered · Which model runs what · What breaks at productization

Companion to: POA v1.1 · App Spec v1 · VaNi Product Framework v2.0 · Proprietary LLM Strategy v1.0

Version 1.1 | 31 July 2026 | Working Document

Author: Charan Kamal Bommakanti, Founder & CEO

Confidential — Vikuna Technologies

1. Purpose & The Two Horizons

VaNiGTM has an orchestrator; VaNi AI adds agents to it. This note fixes how agents are divided, what they may load into context, which model serves which task, and what physically runs where — so that adding the second, third and eighth service does not multiply the architecture.

Every constraint here has two answers, and confusing them is the main risk:

Horizon	Situation	Governing Constraint
H1 — Now	Vikuna-exclusive. One tenant (Vikuna Consulting), a handful of partners, low concurrency, Charan controls every input.	Almost nothing binds. Build for clarity and correctness, not for scale.
H2 — Product	VaNi AI offered to external tenants. Many tenants, unknown concurrency, untrusted inputs, regulated clients.	Isolation, throughput, retrieval quality and compaction all bind at once. §11 lists what breaks and the trigger for each.

The rule that makes H2 cheap

Nothing in H1 needs to be built at H2 scale. But H1 decisions that are cheap now and expensive later must be taken correctly now: agents stateless, tenant_id on every path, retrieval budgets enforced in code, no in-process caches, facts classified by volatility at write time. Those five cost nothing today and are the difference between a two-week and a two-quarter productization.

2. The Decomposition Principle

Split agents by capability and event, never by service line. A service line is something Vikuna sells; an agent is something the platform does in response to an event. Conflating them means every new service grows the architecture — eight services at three agents each is twenty-four agents, each with its own prompts, tests, failure handling and on-call surface.

Candidate	Ruling	Why
“AI-related might be one agent”	Not an agent — a category	Four of the eight services produce identical machine behaviour: score an instrument, generate a report, route a referral. One config-driven Assessment Agent covers all of them. A new service is a new definition row, not a new agent.
“Lead management and lead scoring — one or different”	One agent, two functions	They share the lead entity and the same KnowledgeTree slice. Splitting them creates a coordination problem and buys nothing. Split only when scoring starts calling slow external enrichment — and then as a second event handler inside the same module.
“AI Automation — one agent or merged into the AI agent”	Neither — it is a service, not an agent	AI Automation is delivered to clients. The platform’s agents run Vikuna’s own go-to-market. The only legitimate overlap is that the platform demonstrates the capability being sold.

3. The Agent Roster

A small, stable set that all eight service lines reuse. Agents are registry entries in a shared worker — not separate services (see §6).

Agent	Responds To	Owns	Horizon
Orchestrator	All events	Routing, retries, conflict resolution, dead-letter handling	Exists (VaNiGTM)
Assessment Agent	ASSESSMENT_COMPLETED	Scoring (deterministic), report row, narrative, verdict + referral routing — for ANY assessment definition	v1 — build now
Lead Agent	LEAD_CAPTURED, LEAD_STATUS_CHANGED	ICP scoring, enrichment, referral decisions, status transitions, stale-lead detection	v1.1
Outreach Agent	REPORT_READY, SEQUENCE_STEP_DUE	Email and sequence dispatch, sender hygiene	v1.1 (VaNiGTM has sequence-skill)
Knowledge Agent	KNOWLEDGE_UPDATED	KnowledgeTree maintenance, summarization, compaction	Exists — continues
Research Agent	PROFILE_COMPLETE, manual	Market and competitor research	Later (GTM roadmap Phase 5)

Continuity for v1

POA v1.1 generates the report in-process rather than on the event bus, because no worker is deployed. Build it agent-shaped anyway: a pure function taking an event payload and returning a result, with no dependence on HTTP request context. Registering it on the bus later then costs one line instead of a rewrite.

4. The Three-Tier Test — “should this be agentic?”

Tier	Use when	Examples
1. Config row	Only the content differs; behaviour is identical	A new assessment (Data Audit, V-ORC Readiness), new failure-mode copy, new band thresholds, a new email template
2. Skill function	Deterministic work with known input and output; no autonomous judgement	Scoring, report rendering, CSV import, status transition
3. Agent	Must decide something autonomously in response to an event, or calls a model to do so	Choosing a referral path from mixed signals, composing narrative, deciding whether a lead is worth enriching

Agentic architecture means new features reuse agents — not that each feature gets one. If a proposed agent’s job description contains the name of a service line, it belongs in tier 1.

5. Event Contracts

- Every agent is a pure function of its event payload plus what it retrieves — never of request state, session state, or an in-process cache. This single rule makes horizontal scaling free later.
- Every event carries `tenant_id` and `is_live`. Every query an agent issues is scoped by both, per existing VaNiGTM convention.
- Agents are idempotent: the same event delivered twice produces one outcome. Assume at-least-once delivery.
- Agents never call each other directly — they emit events. Direct calls create a dependency graph that cannot be reordered or retried.
- Prospect-facing paths never block on an agent. Write the deterministic result synchronously; let the agent enrich it afterwards.

6. Memory Budget — RAM

6.1 Main VPS (8 GB, zero swap today)

Component	Approx. RAM	Note
PostgreSQL 17	3–4 GB	Dominant consumer; tune <code>shared_buffers</code> before adding load
PostgREST	~200 MB	Serves other products; out of the VaNi AI path
Nginx	~100 MB	
kd-pipeline-api2	~500 MB	DristiQ
VaNiGTM backend (new)	~400 MB	Express + TypeScript
Worker — ALL agents (new)	~400 MB	One process hosting the whole registry
Projected total	~5.0–5.5 GB	Leaves ~2.5–3 GB headroom

Two hard rules

Add swap before the backend lands. At zero swap on 8 GB, “tight” becomes “OOM-killed” with no warning — and Postgres is the process most likely to be killed.

Agents are registry entries in ONE worker process. Five agents as five Node processes is +2 GB and blows the budget. Splitting workers is a scaling decision that comes with the KVM4 (16 GB) upgrade, never a design default.

6.2 LLM VPS (8 GB)

Qwen3 4B Q4_K_M with 4 parallel slots at 4096 context, alongside n8n, its Postgres, Redis and Traefik. Headroom exists today, but two figures scale and must be measured rather than assumed:

- **KV cache grows linearly with context size × parallel slots.** Raising context from 4096 to 8192 doubles it; doubling slots doubles it again. Before committing, raise the setting and read `llama.cpp`’s own reported allocation — do not estimate.
- **Throughput, not RAM, is the earlier ceiling.** Roughly 5–15 tokens/second on CPU means a 150-word narrative takes 15–40 seconds. Several agents queuing compounds this quickly. Every model call must be asynchronous and fallback-backed.

7. Context Budget

Context, not RAM, is what degrades quality — and it degrades silently. An agent given too much context does not crash; it produces worse output that nobody notices. Three practices, enforced in code rather than by convention:

- **Hard retrieval budget per agent.** A named token ceiling (start at ~1500 tokens of retrieved material) and a top-k node limit, asserted before the call is made — not a guideline in a prompt.
- **Read briefs, not history.** Agents read the rolling brief; only the Knowledge Agent reads raw history.
- **Structured facts as key-values, not prose.** The same information costs three to four times more tokens written as sentences.

On raising the window: context size can and should be increased once measured. But a larger window is not a substitute for retrieval discipline — it raises the ceiling under which the same rules still apply, and every additional token costs generation time on CPU inference.

8. Memory Architecture — The Five Tiers

Memory is not one problem. It is four — what you store, how you select, how you compress, and when you forget — and Vikuna’s domain is unusually well suited to solving most of it without a model in the loop at all. An assessment produces scores, bands, exposures, verdicts and answers; a lead produces company, role, status, partner and timestamps. Almost all of it is typed data, not prose. Most retrieval architectures exist because teams store prose and then have to find it again.

Agents climb these tiers only as far as the task demands:

Tier	Mechanism	Serves	Cost
T0 — Structured facts	SQL columns + JSONB, extracted at write time	Roughly 90% of what agents need: scores, statuses, bands, verdicts, dates	Near zero. A SELECT, ~150 tokens where prose would cost 800
T1 — Rolling brief	One regenerated row per entity, ~200 words	Narrative state of a lead or tenant	One row read; regenerated on change, not per call
T2 — Graph	Nodes + edges in Postgres, recursive CTE traversal	Relationships and multi-hop: which leads share a failure pattern, which partner referred a cluster	Bounded by depth limit and the \$7 budget
T3 — Vector	pgvector over the unstructured residue	Free-text notes; later documents and call transcripts	Top-k only, inside the \$7 budget
T4 — Weights	Fine-tuned model (Vikuna-LLM per LLM Strategy)	Domain memory: house voice, what a Structural band implies, how a failure-mode narrative reads	Zero tokens, forever — paid once at training

The division that resolves the “memory is always an issue” problem

Fine-tuning carries DOMAIN memory; retrieval carries INSTANCE memory. House voice and domain reasoning belong in weights and then cost nothing per call. A specific lead’s facts can never live in weights. Vikuna-LLM-1.0 is therefore a memory strategy as much as a quality one — it attacks the part of the budget retrieval cannot reach.

8.1 On “it is not a real graph”

Postgres with adjacency-list tables and recursive CTEs handles depth-2 and depth-3 traversal over millions of edges perfectly well with the right indexes. What is forgone versus a graph database is native graph algorithms (community detection, centrality) and Cypher ergonomics — not correctness or speed at this scale.

The one place those algorithms will genuinely be wanted is the benchmark ambition in Pilot Pack §8: clustering failure patterns across all clients. That is an analytics workload, not an agent-runtime one — run it as a batch job and materialize the results into columns for agents to read. No graph database required. If query ergonomics ever become the bottleneck, Apache AGE provides Cypher inside Postgres; treat that as a convenience, never as a second datastore.

8.2 Write-time discipline

- **Classify every fact by volatility at write time.** A lead's industry is stable; their status is volatile; their assessment scores are immutable. Retrieval then weights and expires correctly by construction, instead of decay rules being retrofitted later.
- **Confidence score on every auto-extracted node and edge.** The KnowledgeTree is built automatically, which means it has no curator. Retrieval filters on confidence, or extraction noise quietly poisons the well.
- **Provenance on every node.** Where a fact came from and when — without it, no fact can be corrected, expired, or defended to a client asking “how do you know that?”
- **Compaction policy defined now.** Summarize-and-prune by age, redundancy and supersession. Unbounded growth degrades retrieval quality and slows every query; retrofitting compaction over a year of accumulated data is far more expensive than designing it today.

8.3 Two wins available immediately

- **Prefix KV-cache reuse.** llama.cpp supports prompt caching (cache_prompt). Every narrative call shares a long stable prefix — system prompt plus failure-mode copy, easily 1500+ tokens that never vary. Caching it cuts real generation time on the bottleneck that actually binds.
- **Extraction over storage.** Wherever a fact can be a typed column rather than a sentence, make it a column. Every such decision permanently removes tokens from every future prompt.

8.4 Not at H1

Memory-as-a-tool — where the agent calls search_memory() when it decides it needs something — is the most elegant answer and genuinely better in principle. It is not viable yet: each round trip costs 15–40 seconds on CPU inference, and a 4B model is marginal at multi-step tool use. Revisit when a faster tier exists (§9), not before.

9. Model Selection & Tiering

The question is not “Qwen or something else” — it is which model serves which task. The GTM roadmap already settled the mechanism: per-agent model selection, a model override per prompt or agent replacing a single global setting. This note fixes the tiers that mechanism selects between.

Tier	Model	Used for	Status
FAST	A small local model (Qwen3 1.7B class, or a strict-instruction 3B)	Mechanical work: tag extraction, classification, dedup, short deterministic rewrites	Not yet configured — add when a second agent needs it

Tier	Model	Used for	Status
DEFAULT	Qwen3 4B Q4_K_M, self-hosted (current)	Narrative generation, summarization, briefs — short constrained output from supplied facts	Live
ESCALATION	Hosted API (Claude Sonnet per Product Framework \$6)	Genuine multi-step reasoning: research, competitor analysis, ambiguous judgement. Budgeted at under 5% of calls	Architected, not wired

9.1 Why Qwen3 stays the default

- **The task barely needs more.** VaNi AI's narrative is ~150 words, from facts already supplied, in a specified voice, with a template fallback behind it. A 4B model is adequate — arguably generous.
- **Switching the base model costs the flywheel.** The LLM Strategy logs every interaction to fine-tune Qwen3 into Vikuna-LLM-1.0. Changing base model mid-flight devalues the accumulated dataset — which is the one asset in that strategy competitors cannot copy. Add tiers around Qwen3; do not replace it.
- **Licensing is already resolved.** Apache 2.0, commercial use of fine-tuned derivatives permitted — a settled question that a new base model would reopen.

9.2 What actually unlocks the constraint

Throughput is bounded by CPU inference, not by model choice. A larger local model on the same hardware is slower, not better-performing in wall-clock terms. The genuine unlocks, in order of cost: prefix caching (free, §8.3), a FAST tier for mechanical calls (free, config), GPU hosting (real cost, also required by the FLUX/LTX creative plans), and selective escalation to a hosted API for the small share of calls that need real reasoning.

Escalation and DPDP — design for it now

Escalation sends data outside India. Keep escalation payloads PII-free by construction: send scores, mode names and structured context — never respondent identity, email or company. Narrative generation genuinely does not need to know who the person is. Establish this while there is one call site, not fifty.

10. KnowledgeTree Rules

VaNiGTM builds a KnowledgeTree per tenant and per lead, and that continues. The write-time disciplines in §8.2 apply to every node it creates. Two further runtime rules:

- **Retrieve, never inject.** No agent loads a tree. Selection is by vector similarity plus graph proximity, bounded by the §7 budget.
- **Tenant-scoped indexes.** Every retrieval path filtered by tenant_id at the index level, not in application code after the fact.

11. Horizon 2 — What Breaks When This Becomes a Product

None of the following binds while VaNi AI is Vikuna-exclusive. Each has a named trigger; when a trigger fires, that row becomes work.

What breaks	Trigger	What it costs then
App-layer isolation is no longer sufficient	The first external tenant	Partner and tenant isolation currently sit in application code over a BYPASSRLS connection. External tenants require DB-enforced RLS and a non-BYPASSRLS runtime role — the G1 concern, deferred not solved.
CPU inference cannot keep up	~20+ concurrent assessment completions per hour, or any synchronous model path	GPU hosting or a hosted API for the narrative tier, with per-tenant metering.
One worker becomes a bottleneck	Sustained queue depth, or one agent's failure affecting others	Horizontal scaling — free if agents are stateless (§5), a rewrite if they are not.
Retrieval quality decays	KnowledgeTree growth without compaction	Emergency compaction over live data, with quality already degraded in the interim.
Audit and explainability become mandatory	The first regulated tenant (BFSI, healthcare)	Per-decision provenance, model version pinning, retention policy — cheap if provenance exists (§8.2), expensive if it must be reconstructed.
Cost attribution becomes necessary	Any paying external tenant	Per-tenant token metering and quotas — the LLM Strategy already anticipates per-client API keys.

12. Rules for Implementation

- One worker process; agents registered in it. Never a process per agent.
- Agents stateless and idempotent; no in-process caches, no session dependence.
- `tenant_id` and `is_live` on every event and every query, without exception.
- Retrieval budgets asserted in code before any model call; climb the §8 tiers only as far as the task demands.
- Facts classified by volatility, scored for confidence, and stamped with provenance at write time.
- Model is a per-agent parameter, never a global constant. Escalation payloads carry no PII.
- No prospect-facing path blocks on inference; deterministic result first, enrichment after, template fallback always.
- A new service line adds a configuration row. If a change proposes a new agent, apply the §4 test before accepting it.